

Práctica del 10 %

Aplicación de la metodología OOHDM a una plataforma Web

Asignatura	Ingeniería Web
Programa	Ingeniería de Software
Modalidad	Individual
Ponderación	10 %
Entrega	Repositorio de GitHub

Propósito

Aplicar la metodología OOHDM a la plataforma temática desarrollada en la práctica anterior. Cada estudiante deberá modelar el dominio, la navegación, la interfaz abstracta y la implementación, y demostrar que los cuatro modelos corresponden con el funcionamiento real del proyecto publicado en GitHub.

1. Producto esperado

Actualice el repositorio de la plataforma asignada en la práctica del 20 %. El proyecto debe conservar su funcionamiento y agregar una carpeta docs/oohdm con la documentación de las cuatro actividades de OOHDM.

- Un documento docs/oohdm/OOHDM.md que explique los cuatro modelos.
- Cuatro diagramas, uno por cada actividad de OOHDM, exportados en PNG o SVG.
- Los archivos fuente editables de los diagramas, por ejemplo .drawio, .puml o código Mermaid dentro del documento.
- Una sección nueva en README.md que enlace la documentación OOHDM y explique su relación con la plataforma.
- La aplicación descargable y ejecutable mediante node server.js, con la misma base de datos y funcionalidad de la práctica anterior.

2. Alcance de la actividad

La práctica evalúa la aplicación de OOHDM y la correspondencia entre diseño y código. No exige agregar CRUD, autenticación, nuevos formularios ni nuevas tablas. Si los diagramas revelan una inconsistencia con las rutas, los formularios o la base de datos, el estudiante deberá corregirla y documentar el ajuste.

3. Los cuatro modelos obligatorios

3.1 Diseño conceptual

Pregunta que responde: ¿qué información maneja la plataforma y cómo se relaciona?

- Construir un diagrama conceptual de clases con las dos entidades asignadas en la práctica anterior.
- Incluir el nombre de cada clase, sus atributos y la relación entre ellas.
- Indicar las multiplicidades, por ejemplo 1 y 0..*.
- Explicar en un párrafo las reglas del dominio representadas.
- No incluir páginas, rutas HTTP, HTML, CSS ni decisiones visuales en este diagrama.

Archivo sugerido: docs/oohdm/01_modelo_conceptual.png o .svg.

3.2 Diseño navegacional

Pregunta que responde: ¿qué nodos visita el usuario y cómo se desplaza entre ellos?

- Representar los nodos reales de la plataforma: Inicio, Acerca de, Registro, Servicios y las respuestas o confirmaciones.
- Identificar las estructuras de acceso, como el menú o la página desde la cual se localiza una entidad.
- Unir los nodos mediante enlaces o acciones con nombres claros: registrar, guardar, solicitar servicio, volver o consultar.
- Mostrar cómo se conserva o utiliza el identificador de la entidad principal durante el recorrido.
- El diagrama debe corresponder con las rutas GET y POST implementadas.

Archivo sugerido: docs/oohdm/02_modelo_navegacional.png o .svg.

3.3 Diseño de interfaz abstracta

Pregunta que responde: ¿qué información, controles y respuestas contiene cada nodo?

- Representar en un solo diagrama los dos formularios principales: Registro y Servicios.
- Distinguir información visible, campos de entrada, datos ocultos o conservados, botones y respuestas del sistema.
- Indicar cuáles datos escribe el usuario y cuáles genera o completa el servidor.
- Incluir el comportamiento ante un envío correcto y ante datos incompletos.
- No evaluar colores, tipografías o decoración dentro de este modelo; estos elementos pertenecen a la presentación visual.

Archivo sugerido: docs/oohdm/03_interfaz_abstracta.png o .svg.

3.4 Implementación

Pregunta que responde: ¿cómo se materializan los tres modelos anteriores mediante tecnologías concretas?

- Construir un diagrama de implementación o arquitectura que incluya usuario, navegador, servidor Node.js y SQLite.
- Mostrar el intercambio de peticiones GET y POST entre el navegador y el servidor.
- Mostrar las consultas e inserciones realizadas sobre las dos tablas de SQLite.
- Relacionar los nodos con sus rutas, los formularios con sus rutas POST y las clases conceptuales con las tablas.
- Incluir HTML, styles.css, server.js y la base de datos temática.

Archivo sugerido: docs/oohdm/04_implementacion.png o .svg.

4. Correspondencia obligatoria

Después de los cuatro diagramas, incluya una matriz que permita rastrear cada decisión de OOHDM hasta su implementación. Como mínimo debe contener las siguientes columnas:

Elemento OOHDM	Ruta o archivo	Tabla o campo	Evidencia funcional
Entidad principal	Ruta de registro	Primera tabla	Registro almacenado e ID generado
Operación o servicio	Ruta del servicio	Segunda tabla	Operación asociada mediante _id
Nodo de registro	GET /registro y HTML	No aplica	Formulario presentado
Evento de envío	POST asignado	INSERT parametrizado	Respuesta 201 o confirmación

5. Asignación individual de plataformas

Cada estudiante aplicará OOHDM exclusivamente a la plataforma y a las entidades asignadas en la práctica anterior.

Estudiante	Plataforma	Entidad principal	Operación asociada
Mariana Atehortua Grajales	Clínica veterinaria	Mascota	Cita
Cristian Barrientos Torres	Restaurante	Cliente	Reserva
Juan Bedoya Holguin	Evento académico	Participante	Inscripción
Camila Betancur Marinovich	Agencia de viajes	Cliente	Cotización
Anwar Palacios Bejarano	Gimnasio	Usuario	Afiliación
Juan Ramirez Herrera	Hotel	Huésped	Reserva
Jeremy Renteria Serna	Biblioteca	Lector	Préstamo

6. Requisitos de los diagramas

- Los diagramas deben ser elaborados específicamente para la plataforma asignada; no se aceptan diagramas genéricos.
- Cada diagrama debe tener título, nombres legibles, relaciones completas y una explicación clara
- Se permite utilizar diagrams.net, PlantUML, Mermaid, Lucidchart o una herramienta equivalente.
- La documentación debe utilizar exactamente los nombres de entidades, rutas, formularios, tablas y campos presentes en el repositorio.
- Los cuatro diagramas deben visualizarse directamente desde OOHDM.md o desde el README del repositorio.

7. Estructura esperada del repositorio

```
NOMBRE_DEL_REPOSITORIO/
├── index.html
├── acerca.html
├── registro.html
├── servicios.html
├── styles.css
├── server.js
├── README.md
├── proyecto.db
├── docs/
├── oohdm/
│   ├── OOHDM.md
│   ├── 01_modelo_conceptual.png
│   ├── 02_modelo_navegacional.png
│   ├── 03_interfaz_abstracta.png
│   ├── 04_implementacion.png
│   └── fuentes_editables/
```

8. Evidencias mínimas

Evidencia	Qué debe demostrar
Cuatro diagramas	Las cuatro actividades de OOHDM aplicadas a la plataforma.
Matriz de correspondencia	Relación verificable entre modelos, rutas, archivos, tablas y funcionamiento.
Aplicación ejecutándose	Las rutas y formularios descritos en los diagramas funcionan.
Base de datos	Las dos tablas contienen registros relacionados mediante el campo terminado en _id.
Repositorio descargado	Otra persona puede ejecutar node server.js siguiendo el README.

9. Entrega y sustentación

- Entregar únicamente el enlace del repositorio de GitHub en el medio y la fecha indicados.
- El repositorio debe ser accesible, descargable, ejecutable y contener la documentación OOHDM completa.
- La entrega es individual y debe corresponder con la plataforma asignada.
- Durante la sustentación, el estudiante explicará los cuatro diagramas y demostrará su correspondencia con la aplicación.
- El docente podrá seleccionar una ruta, un campo o una tabla y solicitar que se localice en los diagramas y en el código.
- Si se realizaron correcciones a la plataforma después del modelado, deberán explicarse en OOHDM.md.

10. Rúbrica de evaluación

La práctica equivale al 10 % de la asignatura. La calificación se obtiene con la suma de los siguientes criterios.

Criterio	Peso	Superior	Alto	Básico	Insuficiente
Diseño conceptual	20 %	Clases, atributos, relación, multiplicidades y reglas coherentes.	Modelo correcto con omisiones menores.	Relación o atributos incompletos.	No representa el dominio real.
Diseño navegacional	20 %	Nodos, accesos, enlaces, contexto e ID corresponden con las rutas.	Recorrido correcto con detalles menores.	Flujo parcial o poco relacionado con las rutas.	No representa la navegación.
Interfaz abstracta	20 %	Representa ambos formularios, datos, controles, eventos y respuestas.	Modelo funcional con omisiones menores.	Solo representa parte de la interacción.	Confunde interfaz abstracta con diseño visual o no la presenta.
Implementación	15 %	Arquitectura completa y correspondencia correcta con HTML, CSS, Node.js y SQLite.	Representación correcta con un componente incompleto.	Arquitectura parcial o conexiones imprecisas.	No corresponde con la aplicación.
Trazabilidad y coherencia	15 %	Los cuatro modelos, matriz, rutas, tablas y aplicación coinciden completamente.	Existen inconsistencias menores.	Varias decisiones no pueden comprobarse.	Los diagramas son genéricos o contradicen el proyecto.
Repositorio y documentación	5 %	Repositorio accesible, organizado, ejecutable y bien documentado.	Presenta una omisión menor.	Requiere ajustes para consultar o ejecutar.	Enlace inaccesible o archivos faltantes.
Sustentación	5 %	Explica y localiza cada decisión en diagramas y código.	Explica con dudas menores.	Explicación fragmentaria.	No sustenta o no demuestra comprensión.

11. Lista de verificación

- ☐ La documentación corresponde con la temática asignada.
- ☐ El modelo conceptual incluye clases, atributos, relación y multiplicidades.
- ☐ El modelo navegacional contiene nodos, estructuras de acceso y enlaces.
- ☐ El modelo de interfaz abstracta representa los dos formularios y sus respuestas.
- ☐ El diagrama de implementación contiene navegador, Node.js, rutas HTTP y SQLite.
- ☐ Los nombres de rutas, archivos, tablas y campos coinciden con el código.
- ☐ OOHDM.md contiene las explicaciones y la matriz de correspondencia.
- ☐ Los cuatro diagramas se visualizan en el repositorio y tienen fuente editable.
- ☐ La aplicación continúa funcionando y almacena registros en ambas tablas.
- ☐ El README permite descargar, ejecutar y comprobar el proyecto.

12. Resultado esperado de aprendizaje

Al finalizar la práctica, el estudiante deberá justificar cómo una aplicación Web pasa de la representación del problema al recorrido del usuario, de allí a la definición de la interfaz y finalmente a su implementación con HTML, CSS, Node.js, HTTP y SQLite.